

TIMERS and COUNTERS

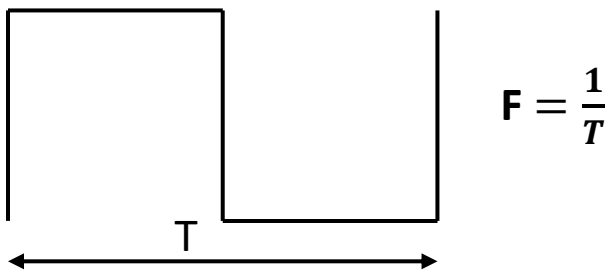
Many applications need to

- ☐ **Count** an Event
- ☐ Generate **Time Delays**

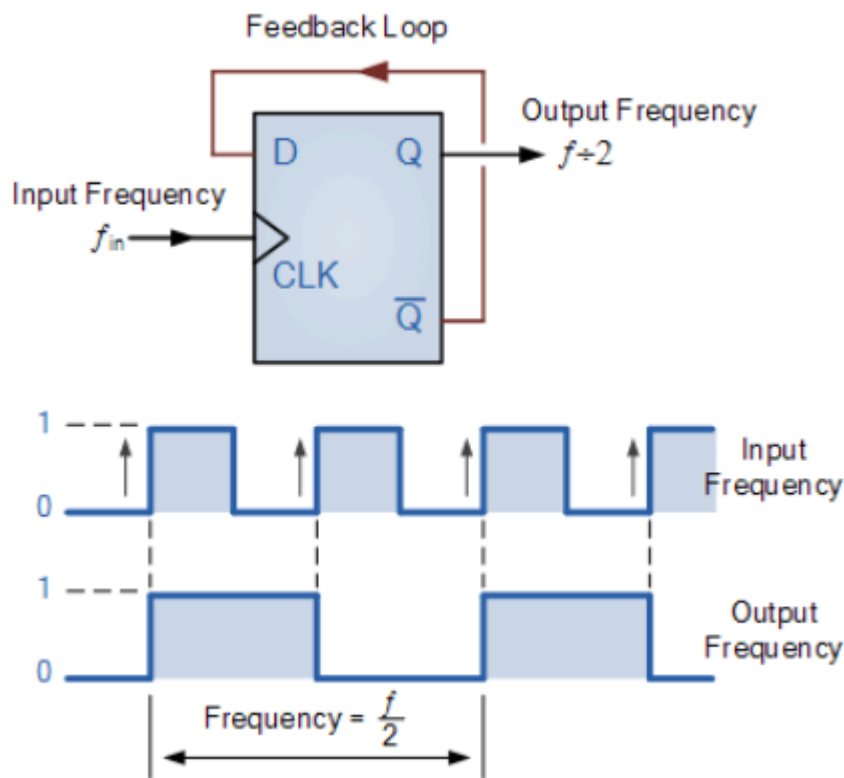
The AVR MEGA 2560 has **6 Timer/Counter registers** (counter register) of **8-bit** and **16-bit**. They are:

8-bit	16-bit
1. Timer/Counter0	1. Timer/Counter1
2. Timer/Counter2	2. Timer/Counter3
	3. Timer/Counter4
	4. Timer/Counter5

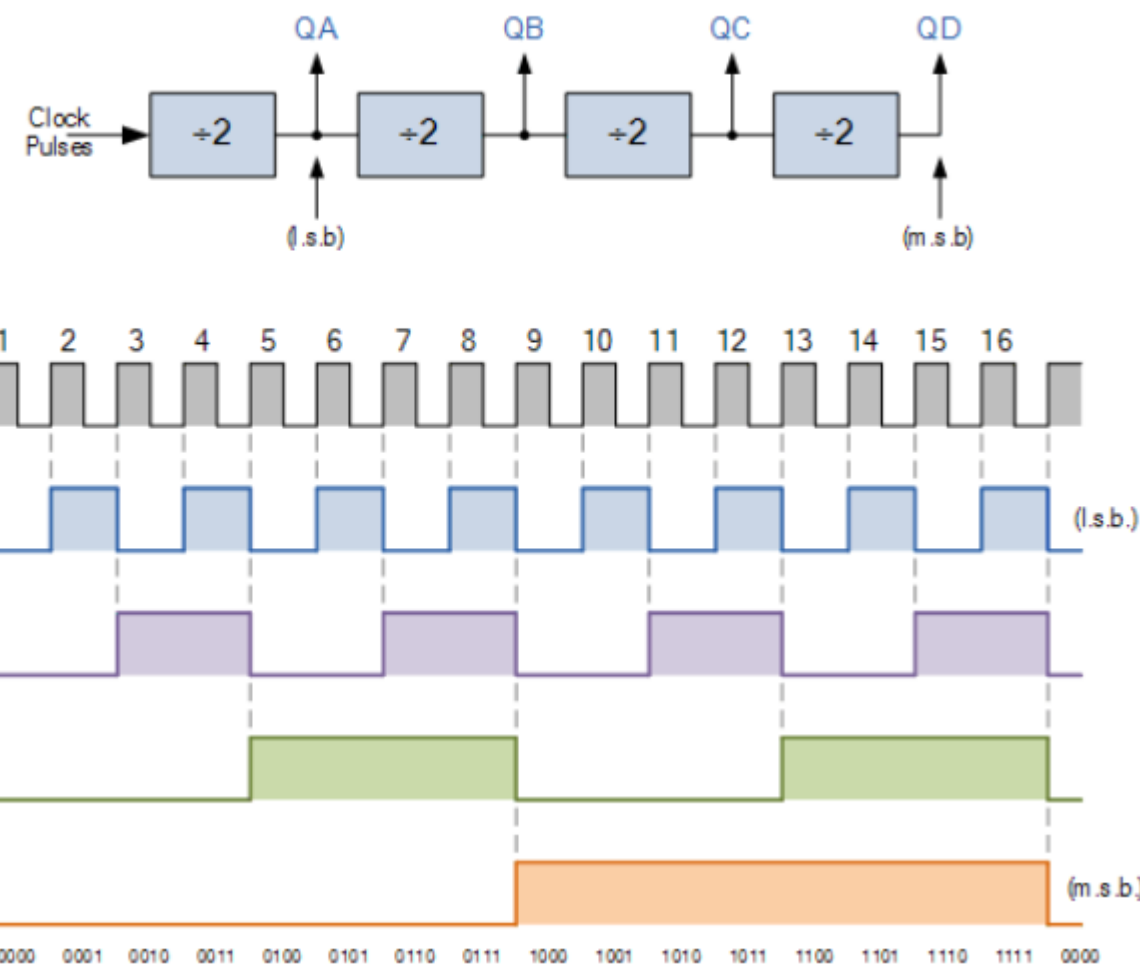
Frequency divider also called a clock divider or scaler or prescaler



Divide-by-2 Counter



4-bit Modulo-16 Counter



Timers:

Two Modes for Timer/Counter

Mode of Operation-1 (Normal Mode)	Mode of Operation-2 (CTC Mode)
✓ Load the counter register, TCNT0 = value ✓ Apply clock to Timer/Counter register ✓ Wait until the counter register overflows ✓ And the flag is SET <u>Example:</u> Consider Oscillator Frequency = 1 MHz with 8-bit register. Content of counter register increment $1/1\text{MHz} = 1\mu\text{s}$. So, if we want a time delay of $3\mu\text{s}$: ✓ Load the Timer/Counter register with 0xFD ✓ ON the Timer/Counter ✓ Wait until flag is set after 3 ticks b/c of overflow ✓ Flag should be cleared by the software (programmer) With First Tick: content of register increment to 0xFE, Second Tick: 0xFF Third Tick: Overflow (0x00) and flag is SET	✓ Clear the counter register TCNT0 = 0 & OCR0A = value ✓ Apply clock to Timer/Counter register ✓ Wait until the counter reaches a number ✓ And the flag is SET <u>Example:</u> Consider Oscillator Frequency = 1 MHz with 8-bit register. Content of counter register increment $1/1\text{MHz} = 1\mu\text{s}$. So, if we want a time delay of $3\mu\text{s}$: ✓ Clear the Timer/counter, load OCR0A=2 ✓ ON the Timer/Counter ✓ Wait until Timer/counter becomes equal to 2 ✓ Flag is set when counter register overflows ✓ Flag should be cleared by the software (programmer)

Programming of Timers:

Every timer needs a **Clock Pulse** to **Tick**. **Clock Source** can be **internal** or **External**.

Internal Clock Source	= Timers
External Clock Source on AVR pins	= Counter

Timer/Counter0 (8-bit) and Timer/Counter1 (16-bit)

[Do by yourself]

Timer/Counter2, Timer/Counter3, Timer/Counter4, Timer/Counter5

8-bit Timer/Counter0

8-bit Registers

TCCR_{nx}, TCNT_n,

OCR_{nx}, TIFR_n,

TIMSK_n

(n = 0 or 2, x = A or B)

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Bits 7:6 – COM0A1:0: Compare Match Output A Mode**

These bits control the Output Compare pin (OC0A) behavior. When OC0A is connected to the pin, the function of the COM0A1:0 bits depends on the WGM02:0 bit setting.

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected
0	1	Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match
1	1	Set OC0A on Compare Match

- Bits 5:4 – COM0B1:0: Compare Match Output B Mode**

These bits control the Output Compare pin (OC0B) behavior. When OC0B is connected to the pin, the function of the COM0B1:0 bits depends on the WGM02:0 bit setting.

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected
0	1	Toggle OC0B on Compare Match
1	0	Clear OC0B on Compare Match
1	1	Set OC0B on Compare Match

- Bits 3, 2 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bits 1:0 – WGM01:0: Waveform Generation Mode**

Combined with the WGM02 bit found in the TCCR0B Register, these bits control the counting sequence of the counter, the source for maximum (TOP) counter value, and what type of waveform generation to be used. Modes of operation supported by the Timer/Counter unit are: Normal mode (counter), Clear Timer on Compare Match (CTC) mode, and two types of Pulse Width Modulation (PWM) modes.

Mode	WGM2	WGM1	WGM0	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	TOP	MAX
4	1	0	0	Reserved			
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved			
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Note: 1. MAX = 0xFF
2. BOTTOM = 0x00

TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B			WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – FOC0A: Force Output Compare A**

The FOC0A bit is only active when the WGM bits specify a non-PWM mode.

- **Bit 6 – FOC0B: Force Output Compare B**

The FOC0B bit is only active when the WGM bits specify a non-PWM mode.

- **Bits 5:4 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 3 – WGM02: Waveform Generation Mode**

(already explained in slide-6)

- **Bits 2:0 – CS02:0: Clock Select**

The three Clock Select bits select the clock source to be used by the Timer/Counter

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$ /(No prescaling)
0	1	0	$clk_{I/O}/8$ (From prescaler)
0	1	1	$clk_{I/O}/64$ (From prescaler)
1	0	0	$clk_{I/O}/256$ (From prescaler)
1	0	1	$clk_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge
1	1	1	External clock source on T0 pin. Clock on rising edge

TCNT0 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
0x26 (0x46)	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Timer/Counter Register gives direct access, both for read and write operations, to the Timer/Counter unit 8-bit counter. Writing to the TCNT0 Register blocks (removes) the Compare Match on the following timer clock. Modifying the counter (TCNT0) while the counter is running, introduces a risk of missing a Compare Match between TCNT0 and the OCR0x Registers.

OCR0A – Output Compare Register A

Bit	7	6	5	4	3	2	1	0	
0x27 (0x47)	OCR0A[7:0]								OCR0A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Register A contains an 8-bit value that is continuously compared with the counter value (TCNT0). A match can be used to generate an Output Compare interrupt, or to generate a waveform output on the OC0A pin.

OCR0B – Output Compare Register B

Bit	7	6	5	4	3	2	1	0	
0x28 (0x48)	OCR0B[7:0]								OCR0B
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Register B contains an 8-bit value that is continuously compared with the counter value (TCNT0). A match can be used to generate an Output Compare interrupt, or to generate a waveform output on the OC0B pin.

TIFR0 – Timer/Counter 0 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x15 (0x35)	–	–	–	–	–	OCF0B	OCF0A	TOV0	TIFR0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:3, 0 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 2 – OCF0B: Timer/Counter 0 Output Compare B Match Flag**

The OCF0B bit is set when a Compare Match occurs between the Timer/Counter and the data in OCR0B – Output Compare Register0 B. OCF0B is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF0B is cleared by writing a logic one to the flag. When the I-bit in SREG, OCIE0B (Timer/Counter Compare B Match Interrupt Enable), and OCF0B are set, the Timer/Counter Compare Match Interrupt is executed.

- **Bit 1 – OCF0A: Timer/Counter 0 Output Compare A Match Flag**

The OCF0A bit is set when a Compare Match occurs between the Timer/Counter0 and the data in OCR0A – Output Compare Register0. OCF0A is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF0A is cleared by writing a logic one to the flag. When the I-bit in SREG, OCIE0A (Timer/Counter0 Compare Match Interrupt Enable), and OCF0A are set, the Timer/Counter0 Compare Match Interrupt is executed.

- **Bit 0 – TOV0: Timer/Counter0 Overflow Flag**

The bit TOV0 is set when an overflow occurs in Timer/Counter0. TOV0 is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, TOV0 is cleared by writing a logic one to the flag. When the SREG I-bit, TOIE0 (Timer/Counter0 Overflow Interrupt Enable), and TOV0 are set, the Timer/Counter0 Overflow interrupt is executed. The setting of this flag is dependent of the WGM02:0 bit setting.

TIMSK0 – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6E)	–	–	–	–	–	OCIE0B	OCIE0A	TOIE0	TIMSK0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:3, 0 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 2 – OCIE0B: Timer/Counter Output Compare Match B Interrupt Enable**

When the OCIE0B bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter Compare Match B interrupt is enabled. The corresponding interrupt is executed if a Compare Match in Timer/Counter occurs, that is, when the OCF0B bit is set in the Timer/Counter Interrupt Flag Register – TIFR0.

- **Bit 1 – OCIE0A: Timer/Counter0 Output Compare Match A Interrupt Enable**

When the OCIE0A bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter0 Compare Match A interrupt is enabled. The corresponding interrupt is executed if a Compare Match in Timer/Counter0 occurs, that is, when the OCF0A bit is set in the Timer/Counter 0 Interrupt Flag Register – TIFR0.

- **Bit 0 – TOIE0: Timer/Counter0 Overflow Interrupt Enable**

When the TOIE0 bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter0 Overflow interrupt is enabled. The corresponding interrupt is executed if an overflow in Timer/Counter0 occurs, that is, when the TOV0 bit is set in the Timer/Counter 0 Interrupt Flag Register – TIFR0.

Calculating Delay Length Using Timers

- a) Crystal Frequency
- b) Prescaler factor
- c) C compiler, b/c various C compilers generate different hex code sizes, and the amount of overhead due to instructions varies by compiler (verify using an Oscilloscope).

Steps to Program Timer/Counter0 in Normal Mode:

1. Load the TCNT0 register with the initial value.
2. Load the values into the TCCR0A and TCCR0B registers. When you select the clock source (TCCR0B), the timer/counter starts to count (during sequential execution of code), and each tick causes the content of the timer/counter to increment by 1.
3. Keep monitoring the timer overflow flag (TOV0) to see if it is set. Get out of the loop when TOV0 raised.
4. Stop the timer by disconnecting the clock source (TCCR0B=0x00).
5. Clear the TOV0 flag for the next round.
6. Go back to step 1 to load TCNT0 again.

Find the value to load into the timer:

Assume that we know the amount of timer delay and to find the value to load in register TCNT0.
Here are the following steps:

1. Calculate Period = $T = 1 / F$ (with no prescaler)
[For prescaler mode = $F / \text{prescaler value}$, $T = 1 / F$]
2. Clocks = Desired Time Delay / T
3. Convert this value into hex
4. Value = $0xFF - \text{Clocks} + 1$
5. Set TCNT0 = Value

Write a program to toggle bit 4 of PORTK continuously every 3125ns. Use Timer0, Normal mode, to generate the delay. Assume XTAL = 16MHz.

$F = 16\text{MHz}$

$T = 1/16\text{MHz} = 62.5\text{ns}$

$\text{Clocks} = \text{Required Time} / T$

$\text{Clocks} = 3125\text{ns} / 62.5\text{ns} = 50 \Rightarrow 0x32$

$\text{Value} = 0xFF - 0x32 + 1 = 0xCE$

```
#include <avr/io.h>
void fordelay()
{
    TCNT0 = 0xCE;           // load TCNT0
    TCCR0A = 0x00;          // Timer0, Normal Mode
    TCCR0B = 0x01;          // No prescaler
    while((TIFR0 & (1<<TOV0)) == 0); // wait for TOV0 to roll over
    TCCR0B = 0x00;          // stop the clock source
    TIFR0 = 0x1<<TOV0;      // clear TOV0
}
int main (void)
{
    DDRK = 0xFF;           // OUTPUT
    while(1)
    {
        PORTK |= (1<<4);
        fordelay();
        PORTK &= ~(1<<4);
        fordelay();
    }
    return 0;
}
```

Write a program to toggle bit 4 of PORTK continuously every 70μs. Use Timer0, Normal mode, and 1:8 prescaler to generate the delay. Assume XTAL = 16MHz.

$F = 16\text{MHz}$

$F/F(\text{Prescaler}) = 16\text{MHz} / 8 = 2\text{MHz}$

$T = 1/2\text{MHz} = 500\text{ns}$

$\text{Clocks} = \text{Required Time} / T$

$\text{Clocks} = 70\mu\text{s} / 500\text{ns} = 140 \Rightarrow 0x8C$

$\text{Value} = 0xFF - 0x8C + 1 = 0x74$

```
#include <avr/io.h>
void fordelay()
{
    TCNT0 = 0x74;           // load TCNT0
    TCCR0A = 0x00;          // Timer0, Normal Mode
    TCCR0B = 0x02;          // 1/8 prescaler
    while((TIFR0 & (1<<TOV0)) == 0); // wait for TOV0 to roll over
    TCCR0B = 0x00;          // stop the clock source
    TIFR0 = 0x1<<TOV0;      // clear TOV0
}
int main (void)
{
    DDRK = 0xFF;           // OUTPUT
    while(1)
    {
        PORTK |= (1<<4);
        fordelay();
        PORTK &= ~(1<<4);
        fordelay();
    }
    return 0;
}
```

Assume XTAL = 16MHz. Write a program to generate a square wave of 64KHz on bit 3 of PORTK continuously.

Look at the following steps:

1. Frequency Required = 64KHz
2. $T_{\text{Required}} = 1/64\text{KHz} = 15.625\mu\text{s}$ the period of square wave
3. $1/2$ of it for the high and low portion of pulse is $7.8125\mu\text{s}$
4. $\text{Clocks} = \text{Required Time} / T = 7.8125\mu\text{s} / 62.5\text{ns} = 125 \Rightarrow 0x7D$
5. $\text{Value} = 0xFF - 0x7D + 1 = 0x83$

Generate Maximum Time Delay and Minimum Time Delay. Assume XTAL = 16MHz.

Assume XTAL = 16MHz. Write a program to generate a square wave of 16KHz on bit 3 of PORTK continuously. (Hint : Use Prescaler)

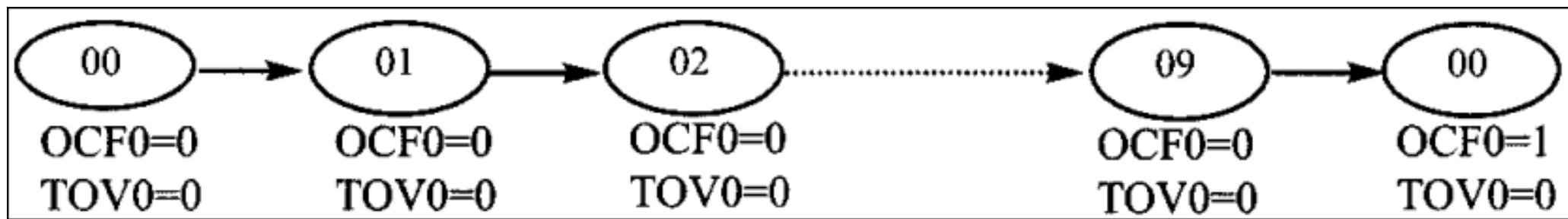
Write a program to generate a square wave of 125Hz on bit 3 of PORTK continuously

Clear Timer on Compare (CTC) Match Mode Programming:

Counts up until the content of the TCNT0 register becomes equal to the content of OCR0A. OCF0A flag will be set when the next clock occur. OCF0A flag is located in the TIFR0 register.

Initialize OCR0A with 0x09

```
void fordelay()
{
    TCNT0 = 0x00;           // load TCNT0
    OCR0A = 0x09;           // load OCR0A
    TCCR0A = 0x02;          // Timer0, CTC Mode
    TCCR0B = 0x01;          // No prescaler
    while((TIFR0 & 0x2) == 0); // wait for OCF0A to roll over
    TCCR0B = 0x00;          // stop the clock source
    TIFR0 = 0x2;            // clear OCF0A
}
```



Assume XTAL = 16MHz. Write a program to generate a delay of 1ms to toggle bit 4 of PORTK continuously. Use Timer0, CTC mode, and with prescaler = 64.

$$F = 16\text{MHz}$$

$$F/F(\text{Prescaler}) = 16\text{MHz} / 64 = 250\text{KHz}$$

$$T = 1/250\text{KHz} = 4\mu\text{s}$$

$$\text{Clocks} = 1\text{ms} / 4\mu\text{s} = 250 \Rightarrow 0\text{xFA}$$

$$\text{Value} = 0\text{xFA} - 1 = 0\text{xF9}$$

```
void fordelay()
{
    TCNT0 = 0x00;           // load TCNT0
    OCR0A = 0xF9;           // load OCR0A
    TCCR0A = 0x02;          // Timer0, CRC Mode
    TCCR0B = 0x03;          // 64 prescaler
    while((TIFR0 & (1<<OCF0A)) == 0); //wait for OCF0A to roll over
    TCCR0B = 0x00;          // stop the clock source
    TIFR0 = (1<< OCF0A);    // clear OCF0A
}
```

Write a program to generate a delay of $750\mu\text{s}$.

If $\text{TCNT0} = 95$ and $\text{OCR0A} = 89$. Find the delay when bit 3 of PORTK get high, and also find, for how much time it stay high.

16-bit Timer/Counter1

16-bit Registers

TCNT_nx, **OCR_nx**

ICR_n(to calculate frequency, duty-cycle, etc.)

8-bit Registers

TCCR_nx, **TIFR_n**, **TIMSK_n**

(**n** = 1,3,4,5 ; **x** = A or B or C)

TCCR1A – Timer/Counter 1 Control Register A

Bit	7	6	5	4	3	2	1	0	
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	COM1C1	COM1C0	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:6 – COMnA1:0: Compare Output Mode for Channel A**
- **Bit 5:4 – COMnB1:0: Compare Output Mode for Channel B**
- **Bit 3:2 – COMnC1:0: Compare Output Mode for Channel C**

The COMnA1:0, COMnB1:0, and COMnC1:0 control the output compare pins (OCnA, OCnB, and OCnC respectively) behavior. If one or both of the COMnA1:0 bits are written to one, the OCnA output overrides the normal port functionality of the I/O pin it is connected to (Same for OCnB, and OCnC).

COMnA1 COMnB1 COMnC1	COMnA0 COMnB0 COMnC0	Description
0	0	Normal port operation, OCnA/OCnB/OCnC disconnected
0	1	Toggle OCnA/OCnB/OCnC on compare match
1	0	Clear OCnA/OCnB/OCnC on compare match (set output to low level)
1	1	Set OCnA/OCnB/OCnC on compare match (set output to high level)

- **Bit 1:0 – WGMn1:0: Waveform Generation Mode**

Combined with the WGMn3:2 bits found in the TCCRnB Register, these bits control the counting sequence of the counter, the source for maximum (TOP) counter value, and what type of waveform generation to be used. Modes of operation supported by the Timer/Counter unit are: Normal mode (counter), Clear Timer on Compare match (CTC) mode, and three types of Pulse Width Modulation (PWM) modes.

Mode	WGMn3	WGMn2 (CTCn)	WGMn1 (PWMn1)	WGMn0 (PWMn0)	Timer/Counter Mode of Operation	TOP	Update of OCRnX at	TOVn Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCRnA	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICRn	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCRnA	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICRn	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCRnA	TOP	BOTTOM
12	1	1	0	0	CTC	ICRn	Immediate	MAX
13	1	1	0	1	(Reserved)			
14	1	1	1	0	Fast PWM	ICRn	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCRnA	BOTTOM	TOP

TCCR1B – Timer/Counter 1 Control Register B

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – ICNCn: Input Capture Noise Canceler**

Setting this bit (to one) activates the Input Capture Noise Canceler. When the Noise Canceler is activated, the input from the Input Capture Pin (ICPn) is filtered. The filter function requires four successive equal valued samples of the ICPn pin for changing its output. The input capture is therefore delayed by four Oscillator cycles when the noise canceler is enabled.

- **Bit 6 – ICESn: Input Capture Edge Select**

This bit selects which edge on the Input Capture Pin (ICPn) that is used to trigger a capture event. When the ICESn bit is written to zero, a falling (negative) edge is used as trigger, and when the ICESn bit is written to one, rising (positive) edge will trigger capture.

- **Bit 5 – Reserved Bit**

This bit is reserved for future use. This bit must be written to zero when TCCRnB is written.

- **Bit 4:3 – WGMn3:2: Waveform Generation Mode**

See TCCRnA Register.

- **Bit 2:0 – CSn2:0: Clock Select**

The three clock select bits select the clock source to be used by the Timer/Counter.

CSn2	CSn1	CSn0	Description
0	0	0	No clock source. (Timer/Counter stopped)
0	0	1	$clk_{I/O}/1$ (No prescaling)
0	1	0	$clk_{I/O}/8$ (From prescaler)
0	1	1	$clk_{I/O}/64$ (From prescaler)
1	0	0	$clk_{I/O}/256$ (From prescaler)
1	0	1	$clk_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on Tn pin. Clock on falling edge
1	1	1	External clock source on Tn pin. Clock on rising edge

TCCR1C – Timer/Counter 1 Control Register C

Bit	7	6	5	4	3	2	1	0	
(0x82)	FOC1A FOC1B FOC1C				–	–	–	–	TCCR1C
Read/Write	W	W	W	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – FOCnA: Force Output Compare for Channel A**
- **Bit 6 – FOCnB: Force Output Compare for Channel B**
- **Bit 5 – FOCnC: Force Output Compare for Channel C**

The FOCnA/FOCnB/FOCnC bits are only active when the WGMn3:0 bits specifies a non-PWM mode. When writing a logical one to the FOCnA/FOCnB/FOCnC bit, an immediate compare match is forced on the waveform generation unit.

The OCnA/OCnB/OCnC output is changed according to its COMnx1:0 bits setting.

- **Bit 4:0 – Reserved Bits**

These bits are reserved for future use. For ensuring compatibility with future devices, these bits must be written to zero when TCCRnC is written.

TCNT1H and TCNT1L – Timer/Counter 1

Bit	7	6	5	4	3	2	1	0	
(0x85)	TCNT1[15:8]								TCNT1H
(0x84)	TCNT1[7:0]								TCNT1L
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The two Timer/Counter I/O locations (TCNTnH and TCNTnL, combined TCNTn) give direct access, both for read and for write operations, to the Timer/Counter unit 16-bit counter.

OCR1AH and OCR1AL – Output Compare Register 1 A

Bit	7	6	5	4	3	2	1	0	
(0x89)	OCR1A[15:8]								OCR1AH
(0x88)	OCR1A[7:0]								OCR1AL
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

OCR1BH and OCR1BL – Output Compare Register 1 B

Bit	7	6	5	4	3	2	1	0	
(0x8B)	OCR1B[15:8]								OCR1BH
(0x8A)	OCR1B[7:0]								OCR1BL
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

OCR1CH and OCR1CL – Output Compare Register 1 C

Bit	7	6	5	4	3	2	1	0	
(0x8D)	OCR1C[15:8]								OCR1CH
(0x8C)	OCR1C[7:0]								OCR1CL
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Registers contain a 16-bit value that is continuously compared with the counter value (TCNTn).

ICR1H and ICR1L – Input Capture Register 1

Bit	7	6	5	4	3	2	1	0	
(0x87)	ICR1[15:8]								ICR1H
(0x86)	ICR1[7:0]								ICR1L
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Input Capture is updated with the counter (TCNTn) value each time an event occurs on the ICPn pin.

TIFR1 – Timer/Counter1 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x16 (0x36)	ICF1				OCF1C	OCF1B	OCF1A	TOV1	TIFR1
Read/Write	R	R	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 5 – ICFn: Timer/Counter_n, Input Capture Flag**

This flag is set when a capture event occurs on the ICPn pin. When the Input Capture Register (ICRn) is set by the WGMn3:0 to be used as the TOP value, the ICFn Flag is set when the counter reaches the TOP value.

- **Bit 3– OCFnC: Timer/Counter_n, Output Compare C Match Flag**

This flag is set in the timer clock cycle after the counter (TCNTn) value matches the Output Compare Register C (OCRnC).

- **Bit 2 – OCFnB: Timer/Counter₁, Output Compare B Match Flag**

This flag is set in the timer clock cycle after the counter (TCNTn) value matches the Output Compare Register B (OCRnB).

- **Bit 1 – OCF1A: Timer/Counter₁, Output Compare A Match Flag**

This flag is set in the timer clock cycle after the counter (TCNTn) value matches the Output Compare Register A (OCRnA).

- **Bit 0 – TOVn: Timer/Counter_n, Overflow Flag**

The setting of this flag is dependent of the WGMn3:0 bits setting. In Normal and CTC modes, the TOVn Flag is set when the timer overflows.

TIMSK1 – Timer/Counter 1 Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6F)	–	–	ICIE1	–	OCIE1C	OCIE1B	OCIE1A	TOIE1	TIMSK1
Read/Write	R	R	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 5 – ICIEn: Timer/Counter, Input Capture Interrupt Enable**

When this bit is written to one, and the I-flag in the Status Register is set (interrupts globally enabled), the Timer/Counter Input Capture interrupt is enabled. The corresponding Interrupt Vector is executed when the ICFn Flag, located in TIFRn, is set.

- **Bit 3 – OCIEnC: Timer/Counter, Output Compare C Match Interrupt Enable**

When this bit is written to one, and the I-flag in the Status Register is set (interrupts globally enabled), the Timer/Counter Output Compare C Match interrupt is enabled. The corresponding Interrupt Vector is executed when the OCFnC Flag, located in TIFRn, is set.

- **Bit 2 – OCIEnB: Timer/Counter, Output Compare B Match Interrupt Enable**

When this bit is written to one, and the I-flag in the Status Register is set (interrupts globally enabled), the Timer/Counter Output Compare B Match interrupt is enabled. The corresponding Interrupt Vector (see “Interrupts” on page 101) is executed when the OCFnB Flag, located in TIFRn, is set.

- **Bit 1 – OCIEnA: Timer/Counter, Output Compare A Match Interrupt Enable**

When this bit is written to one, and the I-flag in the Status Register is set (interrupts globally enabled), the Timer/Counter Output Compare A Match interrupt is enabled. The corresponding Interrupt Vector (see “Interrupts” on page 101) is executed when the OCFnA Flag, located in TIFRn, is set.

- **Bit 0 – TOIE1: Timer/Counter, Overflow Interrupt Enable**

When this bit is written to one, and the I-flag in the Status Register is set (interrupts globally enabled), the Timer/Counter Overflow interrupt is enabled. The corresponding Interrupt Vector (see “Interrupts” on page 101) is executed when the TOVn Flag, located in TIFRn, is set.

Assume XTAL = 16MHz. Write a program to generate a delay of 25.6ms to toggle bit 4 of PORTK continuously. Use Timer1, Normal mode, and with prescaler = ???

```
#include <avr/io.h>
void fordelay()
{
    TCNT1H = 0x38;
    TCNT1L = 0x00;
    TCCR1A = 0x00;                // load TCNT0
    TCCR1B = 0x02;                // Timer0, Normal Mode
    TCCR1C = 0x00;                // No prescaler
    while((TIFR1 & (1<<TOV1)) == 0); // wait for TOV0 to roll over
    TCCR1B = 0x00;                // stop the clock source
    TIFR1 = 0x1;                  // clear TOV0
}
int main (void)
{
    DDRK = 0xFF;                // OUTPUT
    while(1)
    {
        PORTK = 0xFF;
        fordelay();
        PORTK = 0x00;
        fordelay();
    }
    return 0;
}
```

Prescaler	Frequency	Time Period	Clocks	Value (hex)
No Prescaler	16MHz	6.25E-08	409600	64000
(1/8)	2000000	0.0000005	51200	C800
(1/64)	250000	0.000004	6400	1900
(1/256)	62500	0.000016	1600	640
(1/1024)	15625	0.000064	400	190

Assume XTAL = 16MHz. Write a program to generate a delay of 1s to toggle bit 4 of PORTK continuously. Use Timer1, Normal mode, and with prescaler = ???

```
#include <avr/io.h>
void fordelay()
{
    TCNT1H = 0xF4;
    TCNT1L = 0x24;
    TCCR1A = 0x00; // load TCNT0
    TCCR1B = 0x04; // Timer0, Normal Mode
    TCCR1C = 0x00; // No prescaler
    while((TIFR1 & 0x1) == 0); // wait for TOV0 to roll over
    TCCR1B = 0x00; // stop the clock source
    TIFR1 = 0x1; // clear TOV0
}
int main (void)
{
    DDRK = 0xFF; // OUTPUT
    while(1)
    {
        PORTK |= (1<<4);
        fordelay();
        PORTK &= ~(1<<4);
        fordelay();
    }
    return 0;
}
```

Prescaler	Frequency	Time Period	Clocks	Value (hex)
No Prescaler	16MHz	6.25E-08	16000000	F42400
(1/8)	2000000	0.0000005	2000000	1E8480
(1/64)	250000	0.000004	250000	3D090
(1/256)	62500	0.000016	62500	F424
(1/1024)	15625	0.000064	15625	3D09

Counters

AVR timer can also be used to count, detect, and measure the time of events happening outside the AVR. When used as counter, it is a pulse outside the AVR that increments the TCNTn register. CS bits (Clock Selector) in TCCR0B register decide the source of the clock for the timer/counter0. If CS02:00 is set as 6 or 7, the timer is used as counter and gets pulses from a source outside the AVR chip. For timer/counter1, CS bits (Clock Selector) in TCCR1B register decide the source of the clock. If CS12:10 is set as 6 or 7, the timer is used as counter and gets pulses from a source outside the AVR chip. For **timer/counter0 external clock input pin is T0 (PD7/pin-38)**, and for **timer/counter1 external clock input is T1 (PD6/pin-47)**.

Timer/Counter0 (As counter):

Find the value for TCCR0A and TCCR0B if we want to program Timer/Counter0 as Normal mode counter. Use external clock for the clock source and increment on the positive edge.

TCCR0A = 0x00 and TCCR0B = 0x07 Normal, external clock source, no prescaler

Assume that a button input is fed into pin T0, and display the counter on PORTK.

```
#include<avr/io.h>
int main (void)
{
    DDRD &= ~(1<<7);
    PORTD = 0x80; //activate pull-up of
    DDRK = 0xFF;
    TCCR0A = 0x00;
    TCCR0B = 0x07;
    while(1)
    {
        do
        {
            PORTK = TCNT0;
        }while((TIFR0 & (0x1 << TOV0)) == 0); //wait for TOV0 to roll over
        TIFR0 = 0x1 << TOV0;
    }
    return 0;
}
```

Assume that a 1Hz clock pulse is fed into pin T0, use the TOV0 flag to extend Timer/Counter0 to a 16-bit counter and display the counter on PORTF and PORTK.

```
#include<avr/io.h>
int main (void)
{
    DDRD &= ~(1<<7);
    PORTD = 0x80; //activate pull-up of
    DDRF = 0xFF;
    DDRK = 0xFF;
    TCCR0A = 0x00;
    TCCR0B = 0x07;
    while(1)
    {
        do
        {
            PORTF = TCNT0;
        }while((TIFR0 & (0x1 << TOV0)) == 0); //wait for TOV0 to roll over
        TIFR0 = 0x1 << TOV0;
        PORTK++;
    }
    return 0;
}
```

Assume that a 50Hz clock pulse is fed into pin T5, write a program for counter5 in rising edge mode to count the pulses and display the TCNT5H and TCNT5L registers on PORTF and PORTK.

```
#include<avr/io.h>
```

```
int main (void)
```

```
{
```

```
    PORTL = 0x04; //activate pull-up of
```

```
    DDRF = 0xFF;
```

```
    DDRK = 0xFF;
```

```
    TCCR5A = 0x00;
```

```
    TCCR5B = 0x07;
```

```
    TCCR5C = 0x00;
```

```
    while(1)
```

```
    {
```

```
        do
```

```
        {
```

```
            PORTF = TCNT5L;
```

```
            PORTK = TCNT5H;
```

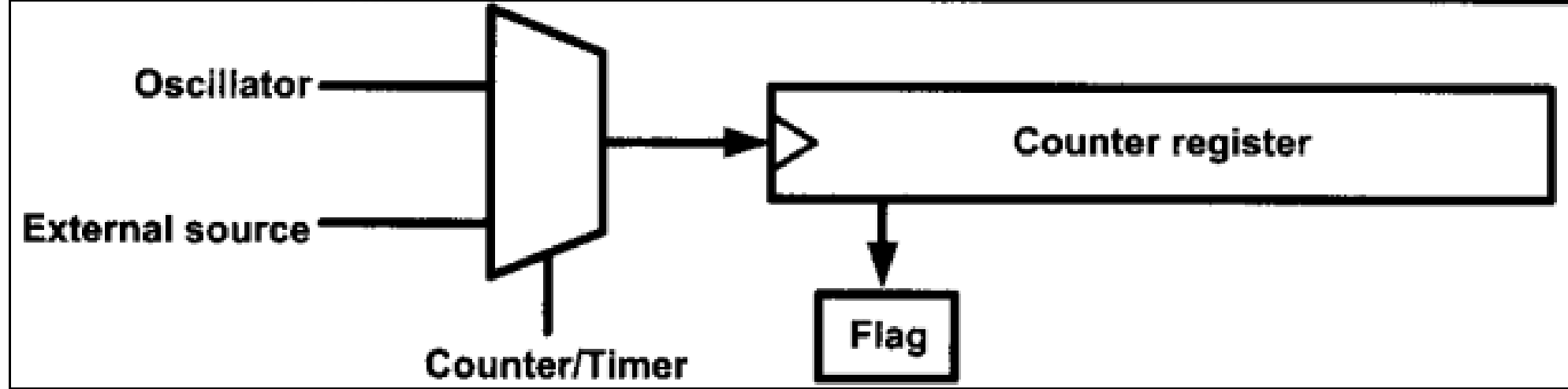
```
        }while((TIFR5 & (0x1 << TOV5)) == 0); //wait for TOV5 to roll over
```

```
        TIFR5 = 0x1 << TOV5;
```

```
    }
```

```
    return 0;
```

```
}
```



Counter:

Basic steps for **Counter Register** in counting an event:

- ✓ Connect **External Event Source** to the associated **Clock Pin** of **counter register**
- ✓ When **event occurs externally**, the **contents of counter register** is **incremented**